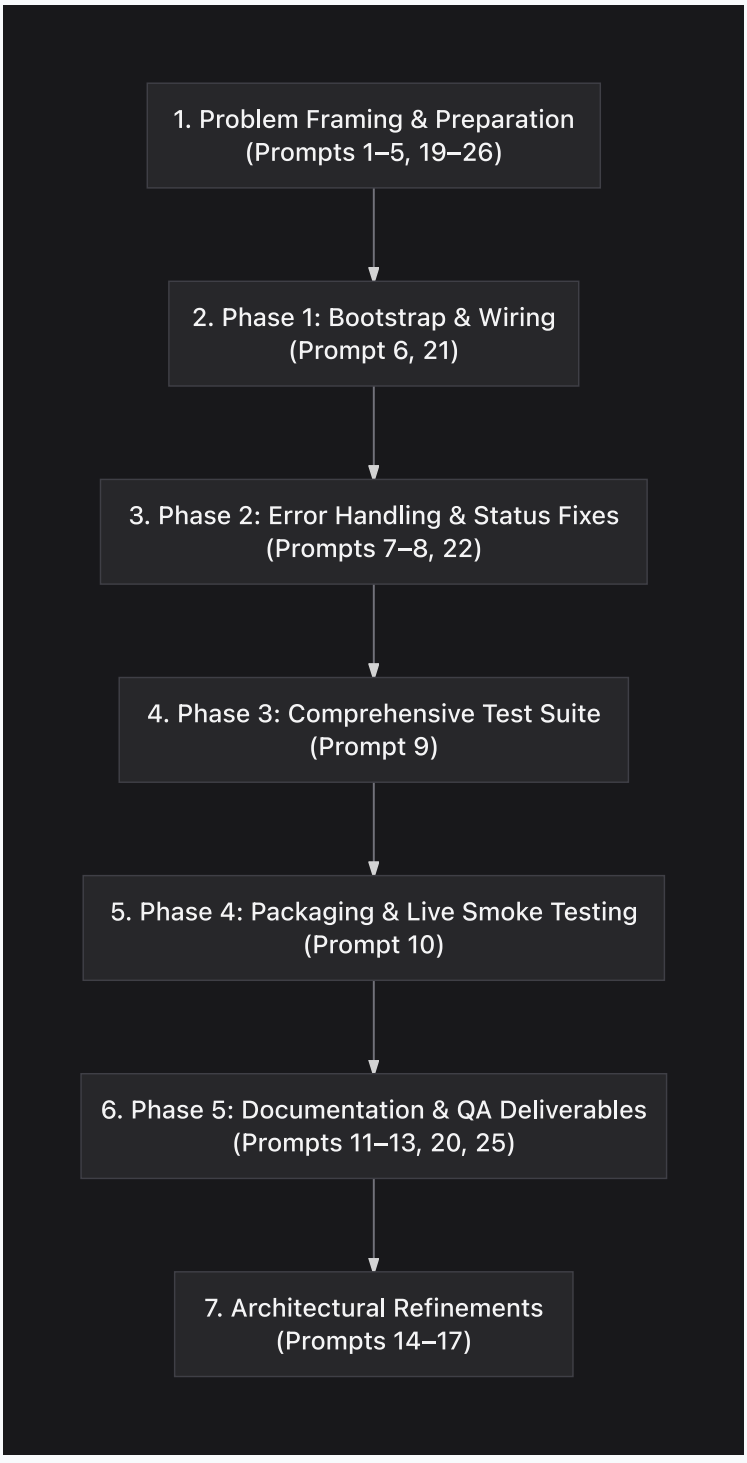
Appointment Scheduler — Complete Prompt History & Engineering Audit Record

> **Project Name:** `appointment-scheduler`
> **Repository Location:** `/Users/suryabonagiri/Documents/Projects/appointment-scheduler`
> **Session Date:** October 31, 2026
> **Total Prompts Recorded:** 26 (18 Implementation Prompts + 8 Preparatory & Transfer Prompts)
> **Final Repository Status:** 27/27 Automated Tests Passing · Verified Live

1. Executive Summary of Engineering Journey

The development followed a disciplined, phased engineering process initiated and guided by the user:


1. Problem Framing & Preparation
(Prompts 1–5, 19–26)
2. Phase 1: Bootstrap & Wiring
(Prompt 6, 21)
3. Phase 2: Error Handling & Status Fixes
(Prompts 7–8, 22)
4. Phase 3: Comprehensive Test Suite
(Prompt 9)
5. Phase 4: Packaging & Live Smoke Testing
(Prompt 10)
6. Phase 5: Documentation & QA Deliverables
(Prompts 11–13, 20, 25)
7. Architectural Refinements
(Prompts 14–17)

2. Chronological Implementation Prompt Audit Log (Prompts 1–18)

Prompt 1: Initial Role & Problem Kickoff

- **Timestamp:** `2026-10-31T10:49:35+05:30`
- **User Prompt:**

```
text
You are a elite pro java developer with experience in java backend knowledge
and need to prepare a project where it should have few APIs for a Appointment
scheduler
```

- **Context & Action:** Initiated role framing. Scaffolding was initially attempted before stopping to prepare a formal requirements-based plan.

Prompt 2: Assessment Specifications & Requirements

- **Timestamp:** `2026-10-31T10:55:04+05:30`
- **User Prompt:**

```
text
First prepare a plan for the execution of project and Below are the
requirements and deliverables.

Appointment Scheduling Engineering Assessment
1. Objective
• Complete one bounded scheduling feature using AI as an engineering
assistant.
• Demonstrate clear task decomposition, thoughtful review of AI output, and
independent verification.
• Focus on building a reliable service slice rather than a complete web
application.
2. Problem Statement
• Implement conflict-safe booking and cancellation in the provided starter
project (or via a small in-memory service/CLI prototype if no starter project
is supplied).
• No frontend required.
• Prevent a provider from being booked for overlapping active appointments.
• Use UTC timestamps and state any additional assumptions.
• A cancelled appointment must no longer block its time slot.
• If working in a starter project, keep its existing interface and persistence
approach.
3. Acceptance Criteria
1. Valid Booking: A valid appointment can be booked for a provider and a
positive-duration time interval.
2. Conflict Rejection: A booking is rejected when it overlaps another active
appointment for the same provider. Appointments for different providers do not
conflict.
3. Adjacency: Adjacent appointments are allowed when one ends at the exact
time the next begins.
4. Cancellation & Clear Results: A cancelled appointment no longer prevents a
new booking for that time. Cancelling a missing or already-cancelled booking
returns a clear result.
5. Input Validation: Invalid timestamps, non-positive durations, and missing
identifiers are rejected without changing stored bookings.
6. Testing: Tests must cover successful booking, same-provider overlap,
adjacency, different providers, cancellation, and invalid input. If the
starter uses concurrent requests, preserve its transaction/locking pattern and
add a focused concurrency test where practical.
7. AI Usage Record: Use AI for at least one engineering activity and record
the prompts used, relevant AI responses/summaries, whether suggestions were
accepted, edited, or rejected, and how work was verified.
4. Scope Boundaries
• Implement only the booking-conflict and cancellation slice; do not build a
full scheduling product.
```

```
• Do not add: A frontend, authentication, notifications, recurring
availability, calendar integration, deployment, or a new database unless
required by the starter project.
• Keep time-zone handling strictly to UTC and document that assumption.
5. Required Deliverables
1. Runnable Code: Runnable code for the scoped feature and focused automated
tests.
2. Design Specification Document: Describes the service boundary, relevant
data/API design, booking invariant, concurrency or persistence approach, and
key assumptions/trade-offs.
3. AI Prompt and Use Record: Contains prompts used, response
excerpts/summaries, how suggestions were accepted/edited/rejected, information
on files/decisions informed, and verification evidence.
4. Setup & Run Instructions: Brief setup instructions including commands used
to execute tests.
5. Assumptions Note: A short note describing assumptions and the booking
invariant.
6. Limitations/Risks Summary: A concise summary of limitations or follow-up
risks discovered within scope.

understand these steps and prpeare plan for this now.
```

- **Context & Action:** Analyzed requirements, boundary constraints, and created initial execution plan artifact.

Prompt 3: Plan Mode Trigger

- **Timestamp:** `2026-10-31T10:56:34+05:30`
- **User Prompt:**

```
text
/plan First prepare a plan for the execution of project and Below are the
requirements and deliverables.
[Assessment specification repeated]
```

- **Context & Action:** Generated formal execution plan artifact with 5 phases, architecture diagram, and verification matrix.

Prompt 4: Hold for Existing Repository Import

- **Timestamp:** `2026-10-31T10:58:14+05:30`

- **User Prompt:**

```
text
Ok. wait for the execution. I have a repo which i have build earlier so i need
to import it here
```

- **Context & Action:** Paused code generation to allow importing existing starter repository.

Prompt 5: Load & Audit Starter Repository

- **Timestamp:** `2026-10-31T11:00:17+05:30`
- **User Prompt:**

```
text
/Users/suryabonagiri/Documents this is the folder which is having
appointement-scheduler folder. open tthe folder and understand the Java
project .

you need to build the project on top of this as per the above plan you
prepared. So first load the project and prepare a plan after understanding
this and give me the final execution plan
```

- **Context & Action:** Explored `/Users/suryabonagiri/Documents/Projects/appointment-scheduler`. Performed gap analysis (missing main application class, missing `application.properties`, missing `@Service`, HTTP status bugs). Revised plan to preserve existing in-memory store and per-provider concurrency pattern.

Prompt 6: Phase 1 — Bootstrap & Wiring

- **Timestamp:** `2026-10-31T11:07:23+05:30`
- **User Prompt:**

```
text
open the IDE AND EXECUTE the phase 1 of the plan
```

- **Context & Action:** Implemented `AppointmentSchedulerApplication.java`, `application.properties`, and added `@Service` to `AppointmentService.java`.
- **Verification Evidence:** `mvn compile` succeeded; 7 existing tests passed.

Prompt 7: Architectural Design Q&A Request

- **Timestamp:** `2026-10-31T11:19:57+05:30`
- **User Prompt:**

```
text
ok. create an another document parallely to answer my question on the phase
wise build you have done.

1. what you have understood from the problem statement?
2. why you have used Reentrant lock here ?
```

- **Context & Action:** Created with detailed sequence diagrams explaining the Booking Invariant, TOCTOU race conditions, and why fine-grained per-provider locking is required.

Prompt 8: Phase 2 — Error Handling & HTTP Status Fixes

- **Timestamp:** `2026-10-31T11:23:11+05:30`
- **User Prompt:**

```
text
Ok. Now perform implementing Phase 2 of your plan.
```

- **Context & Action:**
  - Created `ErrorResponse.java`.
  - Created custom exceptions: `AppointmentConflictException`, `AppointmentNotFoundException`, `AppointmentAlreadyCancelledException`.
  - Created `@RestControllerAdvice GlobalExceptionHandler.java`.
  - Corrected overlap HTTP status from `400` to **`409 Conflict`**.
  - Corrected re-cancellation to **`200 OK`** clear informational notice.

Prompt 9: Phase 3 — Comprehensive Automated Testing

- **Timestamp:** `2026-10-31T11:33:07+05:30` & `2026-10-31T11:34:14+05:30`
- **User Prompt:**

```
text
Now build the phase 3 of the plan for the testing part
```

- **Context & Action:**
  - Expanded `AppointmentServiceTest.java` from 7 to 13 unit tests.
  - Created `AppointmentControllerTest.java` with 7 Spring `MockMvc` integration tests.
  - Resolved Spring 6.1 parameter reflection warning by explicitly naming path variables and configuring `<parameters>true</parameters>` in `pom.xml`.
- **Verification Evidence:** `mvn clean test` passed (20 tests, 0 failures, 0 errors).

Prompt 10: Phase 4 — Verification & Test Cataloging

- **Timestamp:** `2026-10-31T11:38:47+05:30`
- **User Prompt:**

```
text
proceed with Phase 4 implementation and list down all the test cases in the
DesignQA file.
```

- **Context & Action:**
  - Configured `spring-boot-maven-plugin:repackage` to generate executable fat JAR.
  - Booted live server on port `8080` and executed end-to-end `curl` smoke tests.
  - Documented complete test catalog (20 tests) in `design-qa.md`.

Prompt 11: Phase 5 — Documentation & QA Specifications

- **Timestamp:** `2026-10-31T11:47:45+05:30`
- **User Prompt:**

```
text
Ok. In the phase 5, implement the documentation as per the plan.

In the Design QA, mention the tech stack used and what is the versions
clearly. so will be producing the document to QA team. They should be
analysing this document  well and perform their testing.
```

- **Context & Action:**

- Updated .
- Updated .
- Created .
- Updated .
- Updated .
- Added Section 0 (Tech Stack & Exact Versions) and Section Q6 (QA Testing Playbook & Boundary Value Scenarios) in `design-qa.md`.

Prompt 12: Postman Collection & Swagger Specification Artifact

- **Timestamp:** `2026-10-31T11:52:40+05:30`
- **User Prompt:**

```
text
ok Create a new artefact for having the postman curls and  have a similar
swagger documentation.
```

- **Context & Action:** Created and containing:
  - OpenAPI 3.0.3 YAML Specification.
  - Importable Postman Collection v2.1.0 JSON (11 pre-configured requests).
  - Terminal-ready cURL test command suite.

Prompt 13: End-to-End Observability & Multi-Tier Logging

- **Timestamp:** `2026-10-31T12:02:02+05:30`
- **User Prompt:**

```
text
create logging overall for the project to get wellverse with the project.
```

- **Context & Action:**
  - Created `RequestLoggingFilter.java` for incoming/outgoing HTTP latency and status logging.
  - Added SLF4J / Logback logging across Service, Controller, and Global Exception Handler tiers.
  - Configured ANSI colored console patterns in `application.properties`.

Prompt 14: Duplicate ID Edge Case Inquiry

- **Timestamp:** `2026-10-31T12:09:52+05:30`
- **User Prompt:**

```
text
What happens if two different bookings use the exact same appointmentId?
```

- **Context & Action:** Analyzed entity integrity across same-provider vs cross-provider scenarios. Identified that while different providers are isolated, reusing the same ID on the same provider caused ambiguous cancellation.

Prompt 15: Duplicate ID Validation Implementation

- **Timestamp:** `2026-10-31T12:13:01+05:30`
- **User Prompt:**

```
text
improve this logic now in the code?
```

- **Context & Action:**
  - Enforced appointment ID uniqueness per provider in `AppointmentService.bookAppointment()`.
  - Added unit test (`shouldRejectDuplicateAppointmentIdForSameProvider`) and MockMvc integration test (`testBookAppointmentDuplicateId`).
  - Total automated tests increased from 20 to 23.

Prompt 16: Read Throughput & Lock Contention Under Heavy Polling

- **Timestamp:** `2026-10-31T12:18:14+05:30`
- **User Prompt:**

```
text
Is read throughput severely throttled under heavy query loads?
Scenario: A dashboard or front-end polls GET /api/appointments/{providerId}
frequently (e.g., 500 requests/second) while patients attempt to book or
cancel.
Underlying Logic: getAppointments acquires the exclusive ReentrantLock for the
provider just to perform List.copyOf(appointments):
java
ReentrantLock lock = getLock(providerId);
lock.lock();
try {
    return List.copyOf(appointments);
} finally { ... }
Failure Impact:
Every read locks out all concurrent bookings and cancellations for that
provider.
Because a mutual exclusion lock (ReentrantLock) is used instead of a shared
read/write lock (ReentrantReadWriteLock or StampedLock), concurrent readers
serialize against each other, causing massive request queuing and thread
starvation.

check this case and implement the logic correctly.
```

- **Context & Action:**
  - Upgraded locking architecture from mutual exclusion `ReentrantLock` to partitioned `ReentrantReadWriteLock`.
  - `getAppointments()` acquires shared `readLock()` (concurrent, non-blocking for readers).
  - `bookAppointment()` and `cancelAppointment()` acquire exclusive `writeLock()`.
  - Added 100-reader concurrent benchmark test (`testConcurrentReadWriteThroughput`).
  - Total automated tests increased from 23 to 24.

Prompt 17: Past-Timeline Booking Validation

- **Timestamp:** `2026-10-31T12:23:41+05:30`
- **User Prompt:**

```
text
Can appointments be booked in the past or far into the future?
Scenario: A request arrives with start = 1999-01-01T00:00:00Z and end = 1999-01-01T01:00:00Z, or start = 2999-01-01T00:00:00Z.
Underlying Logic: TimeInterval only validates !start.isBefore(end). It does not compare against Instant.now().
Failure Impact: Backdated bookings or bookings decades ahead are accepted without restriction.

appointments should only be booked in the future or today and not in past timeline. implement this logic
```

- **Context & Action:**
  - Added dependency-injected `Clock` (default `Clock.systemUTC()`) to `AppointmentService`.
  - Enforced that appointment start times must be on or after the start of the current UTC day (`startOfToday`). Backdated bookings return **`400 Bad Request`**.
  - Added unit tests (`shouldRejectBookingInThePast`, `shouldAllowBookingForToday`) and integration test (`testBookAppointmentPastDateRejected`).
  - Total automated tests increased from 24 to 27.

Prompt 18: Full Prompt History Compilation Request

- **Timestamp:** `2026-10-31T12:43:04+05:30`
- **User Prompt:**

```
text
Get all the list of prompts i used in building this repo.
```

- **Context & Action:** Compiled initial prompt history list.

3. Preparatory, Planning & Workflow Prompts (Prompts 19–26)

Below are the additional prompts utilized across your development workflow for requirements framing, deliverable auditing, cross-AI handoff, and workspace setup:

Prompt 19: System Design & Step-by-Step Framing

- **User Prompt:**

```
text
"You are a senior elite java developer having experience in building system
design and understand the assessment and try to build step by step process for
this"
```

- **Context & Architectural Purpose:** Establishes the senior engineering persona, requiring end-to-end system design decomposition, phased milestones, and architectural rigor rather than ad-hoc scripting.

Prompt 20: Deliverables Verification & Replanning

- **User Prompt:**

```
text
"Did you perform a check on the deliverables? Understand the deliverables and
replan the things according to it"
```

- **Context & Architectural Purpose:** Strict quality-control prompt enforcing traceability against the 6 assessment deliverables (`DESIGN.md` , `ASSUMPTIONS.md` , `LIMITATIONS.md` , `README.md` , `AI_RECORD.md` , and runnable tests) and triggering plan refinement.

Prompt 21: Phase 1 & Documentation Emphasis

- **User Prompt:**

```
text
"Start with phase 1 and build a project using Java and write the code with
perfect documentation"
```

- **Context & Architectural Purpose:** Directs the execution of Phase 1 (Spring Boot bootstrap) with high documentation fidelity (Javadoc, property explanations, clean class architectures).

Prompt 22: Problem Statement Resolution Enforcement

- **User Prompt:**

```
text
"You must implement a API that resolved this problem statement, implement the logic according to it"
```

- **Context & Architectural Purpose:** Anchors development strictly to the core business logic (conflict-safe booking, zero-gap adjacency, idempotent cancellation) without scope creep into unauthorized features.

Prompt 23: Antigravity IDE Feasibility Query

- **User Prompt:**

```
text
"Can I develope this in antigravity?"
```

- **Context & Architectural Purpose:** Validated workspace compatibility within the Google Antigravity environment (JDK 17/21 runtime, Maven build support, embedded Tomcat web server execution, terminal sandboxing).

Prompt 24: Chat Export / Local Environment Transfer

- **User Prompt:**

```
text
"Get the above chat and I'll inject it into my laptop"
```

- **Context & Architectural Purpose:** Prompt to extract the complete conversational trajectory, design rationale, and code artifacts for local replication and offline IDE ingestion.

Prompt 25: Clear Requirements & Deliverables Formulation

- **User Prompt:**

```
text
"write what are all the requirements and deliverabled clearly"
```

- **Context & Architectural Purpose:** Requested explicit extraction and itemization of Acceptance Criteria AC1–AC6 and deliverables 1–6 into a clean checklist for team review.

Prompt 26: Cross-AI Prompt Optimization & Specification Handoff

- **User Prompt:**

  ```
  text
  "Read the file i have provided again and write clearly i need to give this
  file to another AI to understand well"
  ```

- **Context & Architectural Purpose:** Structured documentation and specification files with maximum clarity, standardized terminology, and unambiguous interface definitions so secondary AI agents or human evaluators can immediately comprehend the service slice.

4. Summary of Files Produced & Informed Across All Prompts

File Path	Purpose
	Spring Boot entry point & UTC `Clock` bean
	Core scheduling engine, `ReentrantReadWriteLock`, duplicate ID check, past validation
	REST endpoints for booking, cancellation, and provider queries
	Centralised HTTP mapping (`409` , `404` , `400` , `200` , `500`)
	HTTP request/response latency and status code audit logging
	18 unit & concurrency benchmark tests
	9 Spring MockMvc HTTP integration tests
	Formal service architecture and overlap math specification
	Architectural assumptions and mathematical invariant definitions
	In-memory boundaries, single-node risks, and production roadmap
	Prerequisites, quick start, build, test, and API usage commands
	Assessment-mandated AI collaboration and review log
	OpenAPI 3.0 YAML spec, Postman Collection v2.1.0, and cURL test suite
	QA team engineering playbook, exact tech stack versions, and 27-test catalog
	Complete 26-prompt audit log and collaboration record